Introduction

For this assignment which was the lookup of Victorian addresses, a linked list was implemented in Stage 1 and a Patricia tree was implemented in Stage 2. The linked list performs lookups by comparing each key in the dataset, with a time complexity of O(n). This can be compared with the Patricia tree which uses shared prefixes across keys and enables partial matches. The Patricia tree has a time complexity of O(logn). For Victorian addresses, it is hypothesised that the Patricia tree will be more efficient than the linked list, especially for larger datasets given varied dataset sizes, duplicates in data, varied key spaces, and varied prefix length.

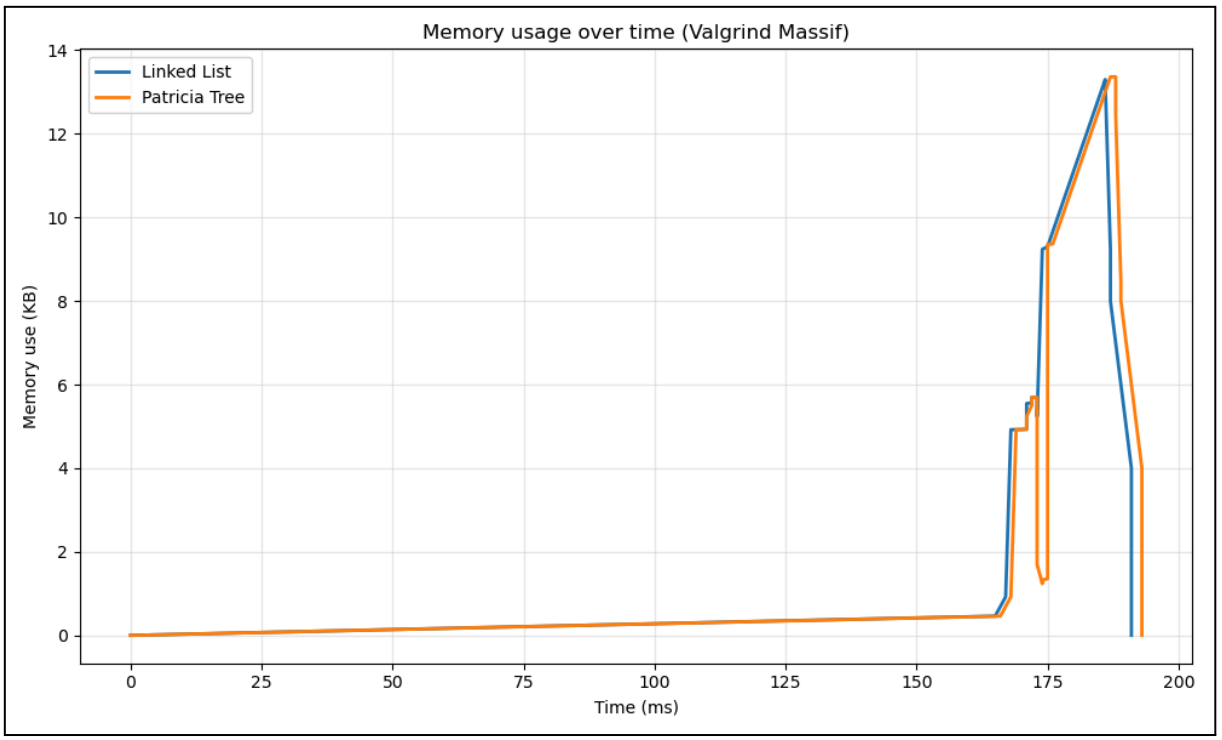
Results

The datasets used were the given datasets as well as generated subsets using head, tail, and sort -R.

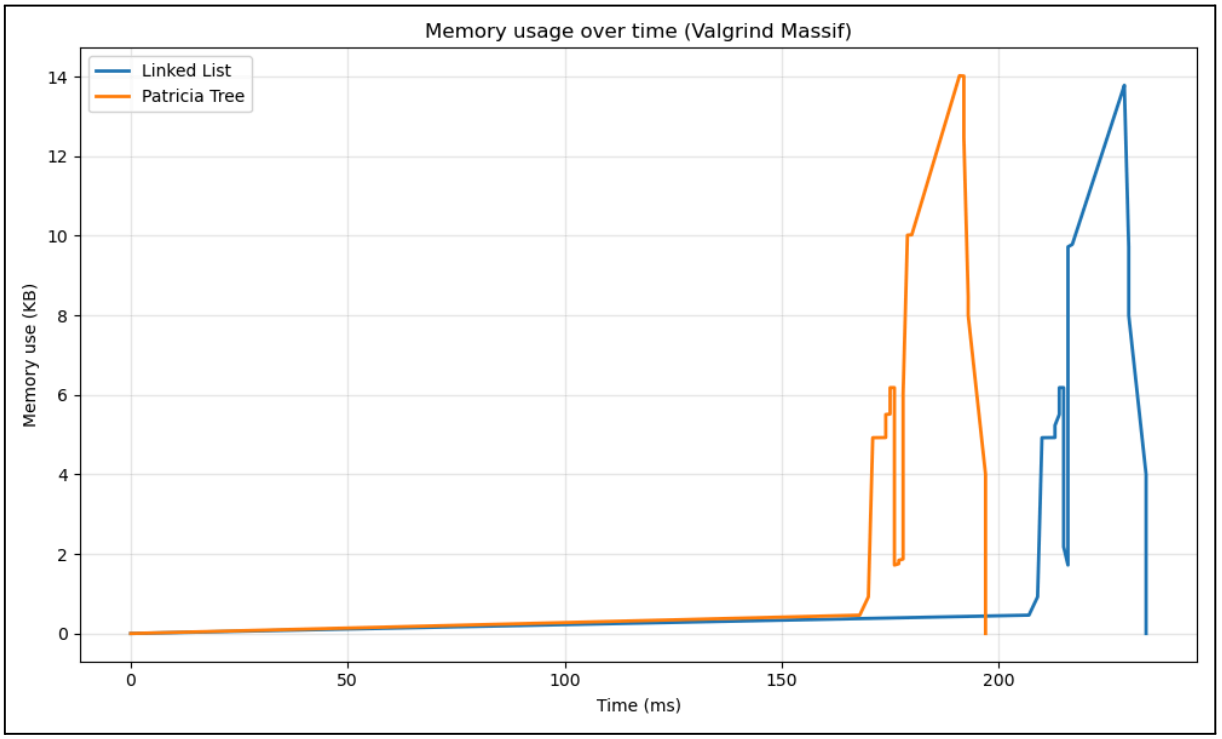
<u>Varying given dataset size</u>

First, graphs were used to visualise time and memory usage for the linked list and Patricia tree for Datasets 1, 2, 22, and 1067.

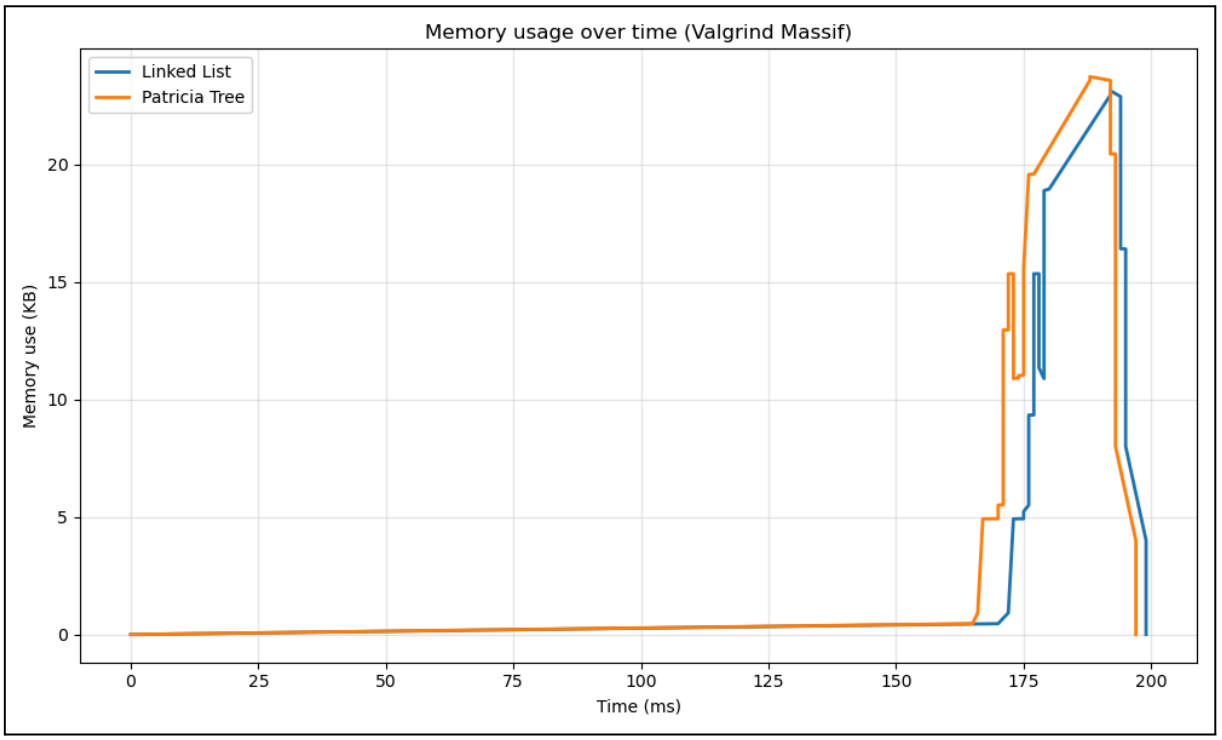
Dataset 1



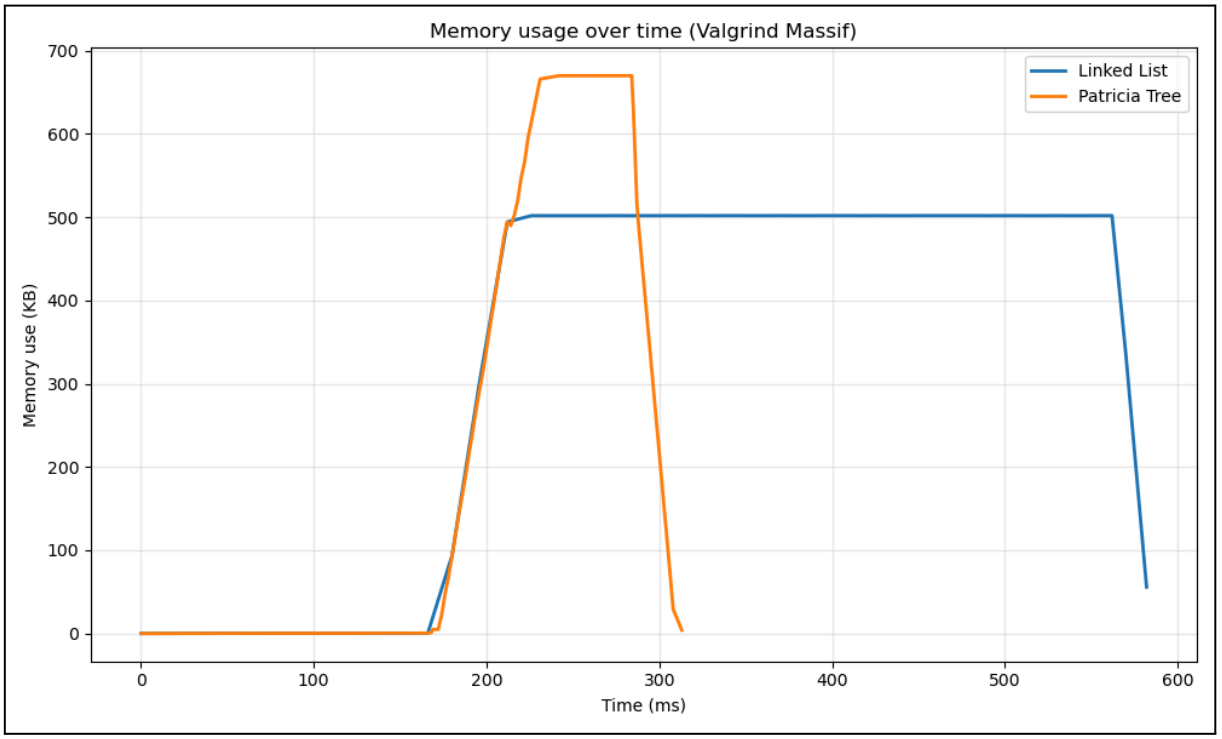
Dataset 2



Dataset 22



Dataset 1067



Referring to the above diagrams, in smaller datasets such as Dataset 1, the data structures perform similarly. Both the linked list and Patricia tree show very similar memory usage patterns and at a smaller scale, the linked list is not costly in comparison to the Patricia tree. In Dataset 2, the Patricia tree peaks slightly earlier and releases memory sooner compared to the linked list, which maintains its memory usage for longer. Thus, the Patricia tree shows efficiency by freeing memory earlier. This contrast can be seen more significantly in Dataset 22, a moderately sized dataset, where the Patricia tree rises faster and releases memory quicker. This is also evident in Dataset 1067, a large dataset, where the linked list plateaus and memory is retained longer whereas the Patricia tree spikes higher but drops rapidly once the query is complete. These show how for larger and increasing datasets, the Patricia tree outperforms the linked list in efficiency.

<u>Varying given dataset size (continued)</u>

These visualisations are accompanied by tables of the average bits, nodes, and strings for each Dataset n where n represents the first n address records.

Dataset 1

Data Structure	**Average bits (b)**	**Average nodes (n)**	**Average strings (s)**
Linked List	272.00	1.00	1.00
Patricia Tree	272.00	1.00	1.00

Dataset 2

Data Structure	Average bits (b)	Average nodes (n)	Average strings (s)
Linked List	282.00	2.00	2.00
Patricia Tree	276.00	2.00	1.00

Dataset 22

Data Structure	Average bits (b)	Average nodes (n)	Average strings (s)
Linked List	2092.33	22.00	22.00
Patricia Tree	274.67	2.67	1.00

Dataset 1067

Data Structure	Average bits (b)	Average nodes (n)	Average strings (s)
Linked List	8480.82	1067.00	1067.00
Patricia Tree	297.16	16.75	1.00

For smaller datasets such as in Dataset 1 and 2, both structures performed similarly. In Dataset 22, the linked list grows in nodes and string comparisons while the Patricia tree remains almost constant. For Dataset 1067, this trend continues where the linked list grows in bit comparisons. The linked list requires over 8000 average bit comparisons and needs to traverse each node whereas the Patricia tree only needs 290 average bit comparisons and around an average of 17 nodes.

<u>Varying dataset size for subsets</u>

The size of the files were further varied in addition to the datasets given. Below are the average bits, nodes, and strings for each Dataset n where n represents the first n address records.

Dataset 10

Data Structure	Average bits (b)	Average nodes (n)	Average strings (s)
Linked List	78.60	10.00	10.00
Patricia Tree	11.60	4.40	1.40

Dataset 100

Data Structure	Average bits (b)	Average nodes (n)	Average strings (s)
Linked List	692.40	100.00	100.00
Patricia Tree	16.80	10.80	1.00

Dataset 1000

Data Structure	Average bits (b)	Average nodes (n)	Average strings (s)
Linked List	7414.80	1000.00	1000.00
Patricia Tree	22.00	15.60	1.80

These support the results in the previous tests with the given datasets of Datasets 1, 2, 22, and 1067 as the larger the dataset, the less bit comparisons and nodes required for the Patricia tree compared to the linked list, and thus better performance.

<u>Adding duplicates</u>

In addition, duplicates were added to produce a dataset of 1000 address records with 50 unique addresses that were repeated 20 times each in the dataset.

Dataset of 1000 addresses with duplicates

Data Structure	Average bits (b)	Average nodes (n)	Average strings (s)
Linked List	13316.33	1000.00	1000.00
Patricia Tree	296.98	7.86	1.00

As shown, the Patricia tree outperforms the linked list as it compresses them under shared prefixes while the linked list must search through strings repeatedly.

<u>Varying key spaces</u>

Further, the linked list and Patricia tree were compared to see the effect of varied key spaces. This was tested by comparing the effect of when many keys shared long prefixes (dense) and when there were keys spread across many prefixes (diverse).

Dataset (dense)

Data Structure	Average bits (b)	Average nodes (n)	Average strings (s)
Linked List	1545.57	189.00	189.00
Patricia Tree	98.13	10.10	1.00

Dataset (diverse)

Data Structure	Average bits (b)	Average nodes (n)	Average strings (s)
Linked List	7771.20	1000.00	1000.00
Patricia Tree	98.00	15.33	1.00

For dense keys, the Patricia tree has fewer string comparisons through compression as there are longer shared prefixes. The Patricia tree also outperforms the linked list too for diverse

keys as it does not have to traverse all records, and only the bits needed to navigate to a mismatch or leaf node. The linked list, regardless of density of key spaces, must traverse the full dataset and thus, are less efficient.

Varying prefix length

Then varying prefix lengths were tested where short prefixes had 4 chars and long prefixes had 10 chars.

Dataset with short prefixes (prefix of length 4 chars)

Data Structure	**Average bits (b)**	**Average nodes (n)**	**Average strings (s)**
Linked List	7763.95	1067.00	1067.00
Patricia Tree	34.70	12.55	1.00

Dataset with long prefixes (prefix of length 10 chars)

Data Structure	**Average bits (b)**	**Average nodes (n)**	**Average strings (s)**
Linked List	7837.65	1067.00	1067.00
Patricia Tree	82.00	12.55	1.00

In both datasets, it is evident that the ability of Patricia trees to compress prefixes allows less comparisons, especially as prefix length increases, whereas linked lists stay linear regardless.

Discussion

In terms of dataset size, when there is a small dataset such as Dataset 1, both the linked list and Patricia tree perform similarly. As the size of the dataset grows, the Patricia tree scales logarithmically, while the linked list scales linearly. Thus, the Patricia tree is more efficient for larger datasets. When there are lots of duplicates, the Patricia tree is also more efficient as it can handle duplicates through shared prefix paths. This is in contrast to the linked list which must compare all entries. For both dense and diverse key spaces, the Patricia tree performs more efficiently compared to the linked list, more so for dense key spaces. When the prefix length is both short (4 chars) and long (10 chars), the Patricia tree performs better than the linked list.

Conclusion

In conclusion, for small datasets there is no significant difference between the Patricia tree and the linked list, especially as the cost of linear traversal is negligible. For larger datasets, and datasets with duplicates, varied key spaces, and varied prefix lengths, the Patricia tree is significantly more efficient. This supports the hypothesis that the Patricia tree is more efficient with O(logn) scaling compared to the linear scaling of the linked list.